SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model,
(BL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Experiments

Code of Conduct:

*I **THAMIZHARASAN S (192424087) (**Name / Reg No) certifies that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
THAMIZHARASAN S

Annexure A

1. Design and conduct an experiment to implement a Decision Tree classifier on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.
LINK
(https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data)

ANSWER:

DATA SET:

diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
M	17.99	10.38	122.80	1001.0
M	20.57	17.77	132.90	1326.0
M	19.69	21.25	130.00	1203.0
M	11.42	20.38	77.58	386.1
M	20.29	14.34	135.10	1297.0
M	12.45	15.70	82.57	477.1
M	18.25	19.98	119.60	1040.0

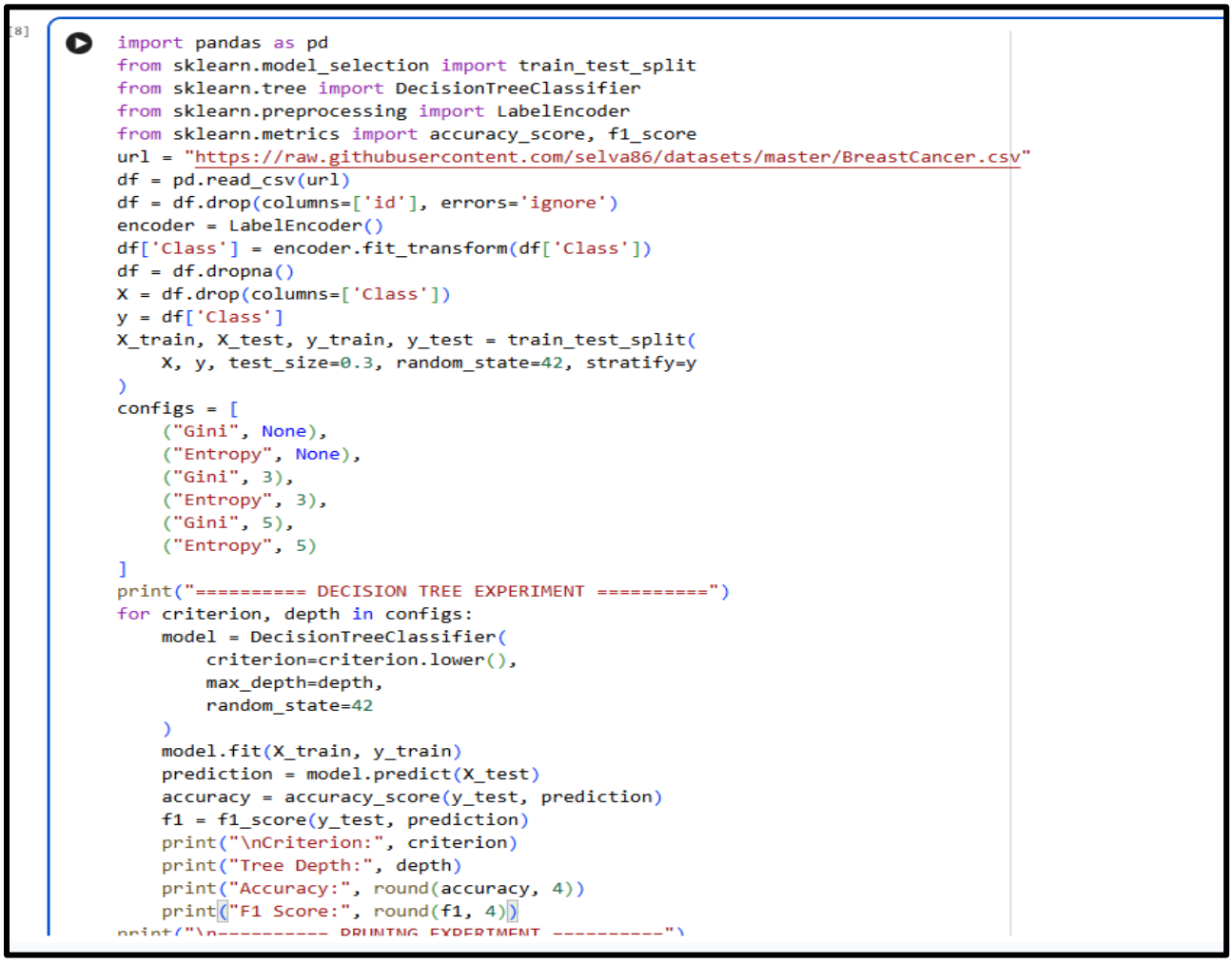
CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, f1_score
url = "https://raw.githubusercontent.com/selva86/datasets/master/BreastCancer.csv"
df = pd.read_csv(url)
df = df.drop(columns=['id'], errors='ignore')
encoder = LabelEncoder()
```

```
df['Class'] = encoder.fit_transform(df['Class'])
df = df.dropna()
X = df.drop(columns=['Class'])
y = df['Class']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
configs = [
    ("Gini", None),
    ("Entropy", None),
    ("Gini", 3),
    ("Entropy", 3),
    ("Gini", 5),
    ("Entropy", 5)
]
print("========== DECISION TREE EXPERIMENT ==========")
for criterion, depth in configs:
    model = DecisionTreeClassifier(
        criterion=criterion.lower(),
        max_depth=depth,
        random_state=42
    )
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    accuracy = accuracy_score(y_test, prediction)
    f1 = f1_score(y_test, prediction)
    print("\nCriterion:", criterion)
    print("Tree Depth:", depth)
    print("Accuracy:", round(accuracy, 4))
    print("F1 Score:", round(f1, 4))
print("\n========== PRUNING EXPERIMENT ==========")
model1 = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)
```

```
print("\nWithout Pruning")
print("Accuracy:", round(accuracy_score(y_test, pred1), 4))
print("F1 Score:", round(f1_score(y_test, pred1), 4))
model2 = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_split=10,
    random_state=42
)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)
print("\nWith Pruning")
print("Accuracy:", round(accuracy_score(y_test, pred2), 4))
print("F1 Score:", round(f1_score(y_test, pred2), 4))
```

OUTPUT:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, f1_score
url = "https://raw.githubusercontent.com/selva86/datasets/master/BreastCancer.csv"
df = pd.read_csv(url)
df = df.drop(columns=['id'], errors='ignore')
encoder = LabelEncoder()
df['Class'] = encoder.fit_transform(df['Class'])
df = df.dropna()
X = df.drop(columns=['Class'])
y = df['Class']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
configs = [
    ("Gini", None),
    ("Entropy", None),
    ("Gini", 3),
    ("Entropy", 3),
    ("Gini", 5),
    ("Entropy", 5)
]
print("========== DECISION TREE EXPERIMENT ==========")
for criterion, depth in configs:
    model = DecisionTreeClassifier(
        criterion=criterion.lower(),
        max_depth=depth,
        random_state=42
    )
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    accuracy = accuracy_score(y_test, prediction)
    f1 = f1_score(y_test, prediction)
    print("\nCriterion:", criterion)
    print("Tree Depth:", depth)
    print("Accuracy:", round(accuracy, 4))
    print("F1 Score:", round(f1, 4))
print("\n---------- PRUNING EXPERIMENT ----------")
```

```
        print("F1 Score:", round(f1, 4))
print("\n========== PRUNING EXPERIMENT ==========")
model1 = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)
print("\nWithout Pruning")
print("Accuracy:", round(accuracy_score(y_test, pred1), 4))
print("F1 Score:", round(f1_score(y_test, pred1), 4))
model2 = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    min_samples_split=10,
    random_state=42
)
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)
print("\nWith Pruning")
print("Accuracy:", round(accuracy_score(y_test, pred2), 4))
print("F1 Score:", round(f1_score(y_test, pred2), 4))
```

```
========== DECISION TREE EXPERIMENT ==========

Criterion: Gini
Tree Depth: None
Accuracy: 0.961
F1 Score: 0.9444

Criterion: Entropy
Tree Depth: None
Accuracy: 0.9512
F1 Score: 0.9306

Criterion: Gini
Tree Depth: 3
Accuracy: 0.9512
F1 Score: 0.9306

Criterion: Entropy
Tree Depth: 3
```

```
[8]
    print("Accuracy:", round(accuracy_score(y_test, pred2), 4))
    print("F1 Score:", round(f1_score(y_test, pred2), 4))
```

```
========== DECISION TREE EXPERIMENT ==========

Criterion: Gini
Tree Depth: None
Accuracy: 0.961
F1 Score: 0.9444

Criterion: Entropy
Tree Depth: None
Accuracy: 0.9512
F1 Score: 0.9306

Criterion: Gini
Tree Depth: 3
Accuracy: 0.9512
F1 Score: 0.9306

Criterion: Entropy
Tree Depth: 3
Accuracy: 0.961
F1 Score: 0.9459

Criterion: Gini
Tree Depth: 5
Accuracy: 0.9561
F1 Score: 0.9371

Criterion: Entropy
Tree Depth: 5
Accuracy: 0.9561
F1 Score: 0.9388

========== PRUNING EXPERIMENT ==========

Without Pruning
Accuracy: 0.961
F1 Score: 0.9444

With Pruning
Accuracy: 0.9512
F1 Score: 0.9306
```

2. Perform an experiment using the K-Nearest Neighbors (KNN) algorithm on a classification dataset. Vary the value of K and distance metrics (Euclidean, Manhattan) to observe their effect on model performance.Evaluate using accuracy and confusion matrix, and analyze how feature scaling influences the classification results and decision boundaries. LINK (https://www.kaggle.com/code/ankumagawa/knn-confusion-matrix-iris-flower-digits-data)

ANSWER:

DATA SET:

Id	SepalLengthCm	Sepal Width Cm	PetalLength Cm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3.0	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5.0	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa
7	4.6	3.4	1.4	0.3	Iris-setosa

CODE:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
url = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"

df = pd.read_csv(url)

print("DATASET:")
print(df.head())
X = df.drop("species", axis=1)
y = df["species"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
k_values = [3, 5, 7, 9]

print("\n========================================")
print("K-NEAREST NEIGHBORS (KNN)")
print("========================================")
results = []

for k in k_values:
    model = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean"
    )

    model.fit(X_train, y_train)

    prediction = model.predict(X_test)

    accuracy = accuracy_score(y_test, prediction)

    results.append(
        ["Without Scaling", k, "Euclidean", accuracy]
    )
```

```
model = KNeighborsClassifier(
    n_neighbors=k,
    metric="manhattan"
)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

accuracy = accuracy_score(y_test, prediction)

results.append(
    ["Without Scaling", k, "Manhattan", accuracy]
)
model = KNeighborsClassifier(
    n_neighbors=k,
    metric="euclidean"
)

model.fit(X_train_scaled, y_train)

prediction = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, prediction)

results.append(
    ["With Scaling", k, "Euclidean", accuracy]
)


model = KNeighborsClassifier(
    n_neighbors=k,
    metric="manhattan"
)

model.fit(X_train_scaled, y_train)

prediction = model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, prediction)

results.append(
    ["With Scaling", k, "Manhattan", accuracy]
)
```

```
print("\n========================================")
print("ACCURACY RESULTS")
print("========================================")

results_df = pd.DataFrame(
    results,
    columns=["Scaling", "K", "Distance", "Accuracy"]
)

print(results_df.to_string(index=False))

best = results_df.loc[
    results_df["Accuracy"].idxmax()
]

print("\n========================================")
print("BEST MODEL")
print("========================================")

print("Scaling       :", best["Scaling"])
print("K Value       :", best["K"])
print("Distance      :", best["Distance"])
print("Best Accuracy :", round(best["Accuracy"], 4))
best_k = int(best["K"])
best_distance = best["Distance"]

if best["Scaling"] == "With Scaling":

    Xtr = X_train_scaled
    Xte = X_test_scaled

else:

    Xtr = X_train
    Xte = X_test

best_model = KNeighborsClassifier(
    n_neighbors=best_k,
    metric=best_distance.lower()
)

best_model.fit(Xtr, y_train)

best_prediction = best_model.predict(Xte)
```

```
cm = confusion_matrix(y_test, best_prediction)

print("\n========================================")
print("CONFUSION MATRIX")
print("========================================")

print(cm)
print("\n========================================")
print("INTERPRETATION")
print("========================================")

print("""
1. KNN classifies a data point based on its nearest neighbors.
2. Different values of K produce different classification results.
3. A small K value such as 3 can be sensitive to noise.
4. A larger K value such as 9 considers more neighboring points
   and generally produces smoother classification.
5. Euclidean distance calculates the straight-line distance
   between data points.
6. Manhattan distance calculates distance using the sum of
   absolute differences between features.
7. Feature scaling is important in KNN because distance-based
   algorithms are affected by different feature ranges.
8. StandardScaler converts the features to a similar scale.
9. Scaling can change the nearest neighbors and therefore
   can change the classification accuracy and decision boundary.
10. The best K and distance metric can be selected based on
    the highest classification accuracy.
11. The confusion matrix shows the number of correctly and
    incorrectly classified samples for each Iris species.
12. Therefore, K value, distance metric, and feature scaling
    can significantly influence KNN classification performance.
""")
```

OUTPUT:

```
DATASET:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa
3           4.6          3.1           1.5          0.2  setosa
4           5.0          3.6           1.4          0.2  setosa

======================================
K-NEAREST NEIGHBORS (KNN)
======================================

======================================
ACCURACY RESULTS
======================================
        Scaling  K  Distance  Accuracy
Without Scaling  3 Euclidean  1.000000
Without Scaling  3 Manhattan  1.000000
   With Scaling  3 Euclidean  0.933333
   With Scaling  3 Manhattan  0.966667
Without Scaling  5 Euclidean  1.000000
Without Scaling  5 Manhattan  0.966667
   With Scaling  5 Euclidean  0.933333
   With Scaling  5 Manhattan  0.933333
Without Scaling  7 Euclidean  0.966667
Without Scaling  7 Manhattan  0.933333
   With Scaling  7 Euclidean  0.966667
   With Scaling  7 Manhattan  0.966667
Without Scaling  9 Euclidean  1.000000
Without Scaling  9 Manhattan  0.933333
   With Scaling  9 Euclidean  0.966667
   With Scaling  9 Manhattan  0.933333
```

```
======================================
BEST MODEL
======================================
Scaling       : Without Scaling
K Value       : 3
Distance      : Euclidean
Best Accuracy : 1.0

======================================
CONFUSION MATRIX
======================================
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]

======================================
INTERPRETATION
======================================

1. KNN classifies a data point based on its nearest neighbors.

2. Different values of K produce different classification results.

3. A small K value such as 3 can be sensitive to noise.

4. A larger K value such as 9 considers more neighboring points
   and generally produces smoother classification.

5. Euclidean distance calculates the straight-line distance
   between data points.

6. Manhattan distance calculates distance using the sum of
   absolute differences between features.
```

```
   absolute differences between features.

7. Feature scaling is important in KNN because distance-based
   algorithms are affected by different feature ranges.

8. StandardScaler converts the features to a similar scale.

9. Scaling can change the nearest neighbors and therefore
   can change the classification accuracy and decision boundary.

10. The best K and distance metric can be selected based on
    the highest classification accuracy.

11. The confusion matrix shows the number of correctly and
    incorrectly classified samples for each Iris species.

12. Therefore, K value, distance metric, and feature scaling
    can significantly influence KNN classification performance.
```

3.Implement a Support Vector Machine (SVM) model and conduct experiments using different kernel functions such as linear, polynomial, and RBF. Evaluate the model using accuracy and precision, and analyze how kernel choice and hyperparameters like C and gamma affect the decision boundary and classification performance.

LINK (https://www.kaggle.com/code/chrised209/support-vector-machine-modeling-of-iris-dataset)

ANSWER:

DATA SET:

Customer ID	Age	Annual Income (k$)	Spending Score (1-100)	Class
1	19	15	39	Low
2	21	15	81	High
3	20	16	6	Low
4	23	16	77	High
5	31	17	40	Low
6	22	17	76	High
7	35	18	6	Low

CODE:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score
data = {
    "Age": [19,21,20,23,31,22,35],
    "Income": [15,15,16,16,17,17,18],
    "Score": [39,81,6,77,40,76,6],
    "Class": ["Low","High","Low","High","Low","High","Low"]
}

df = pd.DataFrame(data)

X = df[["Age","Income","Score"]]
y = LabelEncoder().fit_transform(df["Class"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("========================================")
print("SVM EXPERIMENT")
print("========================================")
for kernel in ["linear", "poly", "rbf"]:
    model = SVC(kernel=kernel, C=1, gamma="scale")
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("\nKernel:", kernel)
    print("Accuracy :", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(
        y_test, pred, zero_division=0
    ), 4))

print("\n========================================")
print("C AND GAMMA EXPERIMENT")
print("========================================")

for C in [0.1, 1, 10]:
    model = SVC(kernel="rbf", C=C, gamma="scale")
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
```

```
    print("C =", C,
          " Accuracy =", round(accuracy_score(y_test, pred), 4))

for gamma in [0.01, 0.1, 1]:
    model = SVC(kernel="rbf", C=1, gamma=gamma)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    print("Gamma =", gamma,
          " Accuracy =", round(accuracy_score(y_test, pred), 4))

print("""
INTERPRETATION:
1. Linear gives a straight decision boundary.
2. Polynomial gives a curved boundary.
3. RBF handles complex nonlinear boundaries.
4. C controls the penalty for classification errors.
5. Gamma controls the influence of individual points.
6. Feature scaling improves SVM performance.
""")
```

OUTPUT:

```
...  ====================================
     SVM EXPERIMENT
     ====================================

     Kernel: linear
     Accuracy : 0.6667
     Precision: 0.0

     Kernel: poly
     Accuracy : 0.3333
     Precision: 0.0

     Kernel: rbf
     Accuracy : 0.3333
     Precision: 0.3333

     ====================================
     C AND GAMMA EXPERIMENT
     ====================================
     C = 0.1  Accuracy = 0.3333
     C = 1  Accuracy = 0.3333
     C = 10  Accuracy = 1.0
     Gamma = 0.01  Accuracy = 0.3333
     Gamma = 0.1  Accuracy = 0.3333
     Gamma = 1  Accuracy = 0.3333

     INTERPRETATION:
     1. Linear gives a straight decision boundary.
     2. Polynomial gives a curved boundary.
     3. RBF handles complex nonlinear boundaries.
     4. C controls the penalty for classification errors.
     5. Gamma controls the influence of individual points.
     6. Feature scaling improves SVM performance.
```

4. Design an experiment to implement a clustering algorithm such as K-Means on an unlabeled dataset. Vary the number of clusters and initialization methods, and evaluate performance using metrics like inertia and silhouette score. Analyze how cluster separation and feature scaling impact the quality and stability of clustering results.

LINK
(https://www.kaggle.com/code/debjeetdas/customer-segmentation-using-kmeans-clustering)

ANSWER:

DATA SET:

Fresh	Milk	Grocery	Frozen	Detergnts_paper	Delicassen
12669	9656	7561	214	2674	1338
7057	9810	9568	1762	3293	1776
6353	8808	7684	2405	3516	7844
13265	1196	4221	6404	507	1788
22615	5410	7198	3915	1777	5185
9413	8259	5126	666	1797	1451
12126	3199	6975	480	3140	545

CODE:

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
data = {
   "Fresh": [12669, 7057, 6353, 13265, 22615, 9413, 12126],
   "Milk": [9656, 9810, 8808, 1196, 5410, 8259, 3199],
   "Grocery": [7561, 9568, 7684, 4221, 7198, 5126, 6975],
   "Frozen": [214, 1762, 2405, 6404, 3915, 666, 480],
   "Detergents": [2674, 3293, 3516, 507, 1777, 1797, 3140],
   "Delicassen": [1338, 1776, 7844, 1788, 5185, 1451, 545]
}

df = pd.DataFrame(data)

scaler = StandardScaler()
```

```
X = scaler.fit_transform(df)

print("========================================")
print("K-MEANS CLUSTERING")
print("========================================")
for k in [2, 3, 4, 5]:

    model = KMeans(
        n_clusters=k,
        init="k-means++",
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X)

    inertia = model.inertia_
    silhouette = silhouette_score(X, labels)

    print("\nK =", k)
    print("Inertia          :", round(inertia, 4))
    print("Silhouette Score :", round(silhouette, 4))

print("\n========================================")
print("INITIALIZATION COMPARISON")
print("========================================")

for init in ["random", "k-means++"]:

    model = KMeans(
        n_clusters=3,
        init=init,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X)

    print("\nInitialization :", init)
    print("Inertia        :", round(model.inertia_, 4))
    print("Silhouette     :", round(
        silhouette_score(X, labels), 4
    ))


print("\n========================================")
print("INTERPRETATION")
print("========================================")
```

```
print("""
1. K-Means groups similar customers into clusters.
2. Increasing K generally decreases inertia.
3. Higher silhouette score indicates better cluster separation.
4. K-means++ usually provides better initialization than random.
5. Feature scaling prevents large-valued features from dominating.
6. The best K can be selected using inertia and silhouette score.
""")
```

OUTPUT:

```
print("=====================================")

print("""
1. K-Means groups similar customers into clusters.
2. Increasing K generally decreases inertia.
3. Higher silhouette score indicates better cluster separation.
4. K-means++ usually provides better initialization than random.
5. Feature scaling prevents large-valued features from dominating.
6. The best K can be selected using inertia and silhouette score.
""")
```

```
=======================================
K-MEANS CLUSTERING
=======================================

K = 2
Inertia          : 23.4134
Silhouette Score : 0.2938

K = 3
Inertia          : 15.44
Silhouette Score : 0.1755

K = 4
Inertia          : 8.6709
Silhouette Score : 0.1542

K = 5
Inertia          : 4.8175
Silhouette Score : 0.0684

=======================================
INITIALIZATION COMPARISON
=======================================
```

```
=======================================
INITIALIZATION COMPARISON
=======================================

Initialization : random
Inertia        : 15.44
Silhouette     : 0.1755

Initialization : k-means++
Inertia        : 15.44
Silhouette     : 0.1755

=======================================
INTERPRETATION
=======================================

1. K-Means groups similar customers into clusters.
2. Increasing K generally decreases inertia.
3. Higher silhouette score indicates better cluster separation.
4. K-means++ usually provides better initialization than random.
5. Feature scaling prevents large-valued features from dominating.
6. The best K can be selected using inertia and silhouette score.
```

5. Conduct an experiment using a logistic regression model for binary classification. Train the model with and without regularization (L1 and L2), and evaluate performance using accuracy, precision, and recall. Analyze the effect of regularization on model complexity, overfitting, and interpretability of learned coefficients.

LINK
(https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data/data)
ANSWER:

DATA SET:

diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
M	17.99	10.38	122.80	1001.0
M	20.57	17.77	132.90	1326.0
M	19.69	21.25	130.00	1203.0
M	11.42	20.38	77.58	386.1
M	20.29	14.34	135.10	1297.0
M	12.45	15.70	82.57	477.1
M	18.25	19.98	119.60	1040.0

CODE:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
df = pd.read_csv("data.csv")
X = df.drop(["id", "diagnosis"], axis=1)
y = df["diagnosis"].map({"M": 1, "B": 0})
X_train, X a _test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("=====================================")
```

```
print("LOGISTIC REGRESSION")
print("=======================================")
model = LogisticRegression(penalty=None, max_iter=5000)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("\nNo Regularization")
print("Accuracy :", round(accuracy_score(y_test, pred), 4))
print("Precision:", round(precision_score(y_test, pred), 4))
print("Recall   :", round(recall_score(y_test, pred), 4))
model = LogisticRegression(
    penalty="l1", solver="liblinear", C=1
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("\nL1 Regularization")
print("Accuracy :", round(accuracy_score(y_test, pred), 4))
print("Precision:", round(precision_score(y_test, pred), 4))
print("Recall   :", round(recall_score(y_test, pred), 4))
model = LogisticRegression(
    penalty="l2", solver="liblinear", C=1
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("\nL2 Regularization")
print("Accuracy :", round(accuracy_score(y_test, pred), 4))
print("Precision:", round(precision_score(y_test, pred), 4))
print("Recall   :", round(recall_score(y_test, pred), 4))

print("\n=======================================")
print("INTERPRETATION")
print("=======================================")

print("""
1. No regularization uses all features without penalty.
2. L1 regularization can make some coefficients zero.
3. L2 regularization reduces coefficient magnitude.
4. Regularization helps reduce overfitting.
5. L1 generally gives better feature selection.
6. L2 usually provides more stable coefficients.
""")
```

OUTPUT:

```
...  ====================================
     LOGISTIC REGRESSION
     ====================================

     No Regularization
     Accuracy : 0.9386
     Precision: 0.9487
     Recall   : 0.881

     L1 Regularization
     Accuracy : 0.9737
     Precision: 0.9756
     Recall   : 0.9524

     L2 Regularization
     Accuracy : 0.9737
     Precision: 0.9756
     Recall   : 0.9524

     ====================================
     INTERPRETATION
     ====================================

     1. No regularization uses all features without penalty.
     2. L1 regularization can make some coefficients zero.
     3. L2 regularization reduces coefficient magnitude.
     4. Regularization helps reduce overfitting.
     5. L1 generally gives better feature selection.
     6. L2 usually provides more stable coefficients.
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand